

Electronic Lab Notebook

Voice-Driven Protocol Guidance & Data Logging System

Requirements Document · Architecture Specification · Demo Script

Version	0.3 — Updated draft
Date	June 2026
Status	In Review
Changes	UPDATED Build tiers: Timer promoted to must-have; AI voice assistant added to stretch NEW Frontend requirements: pages, shortcuts, protocol upload flow UPDATED Stretch features: OCR import + AI voice assistant; SQLite via FastAPI; Deepgram protocols (from v0.2)
Audience	Engineering team, project stakeholders

1. Executive Summary

The Electronic Lab Notebook (ELN) is a voice-driven laboratory assistant that lets researchers keep their hands free and their minds on the experiment. A scientist speaks naturally — loading a protocol, logging an observation, starting a timer, checking an inventory location, or asking what step comes next — and the system transcribes, parses intent, and responds in real time.

The core product is completable and demoable at the must-have tier: voice capture, Deepgram nova-2 speech-to-text transcription, LLM intent routing, SQLite logging via FastAPI, protocol guidance, and timers. Inventory query and advanced features follow in subsequent tiers.



Demo constraint:

The 30-second pitch requires zero keyboard or mouse interaction. All five demo beats are satisfied by voice alone. This is the non-negotiable design requirement that drives every interface decision.

2. Build Tiers

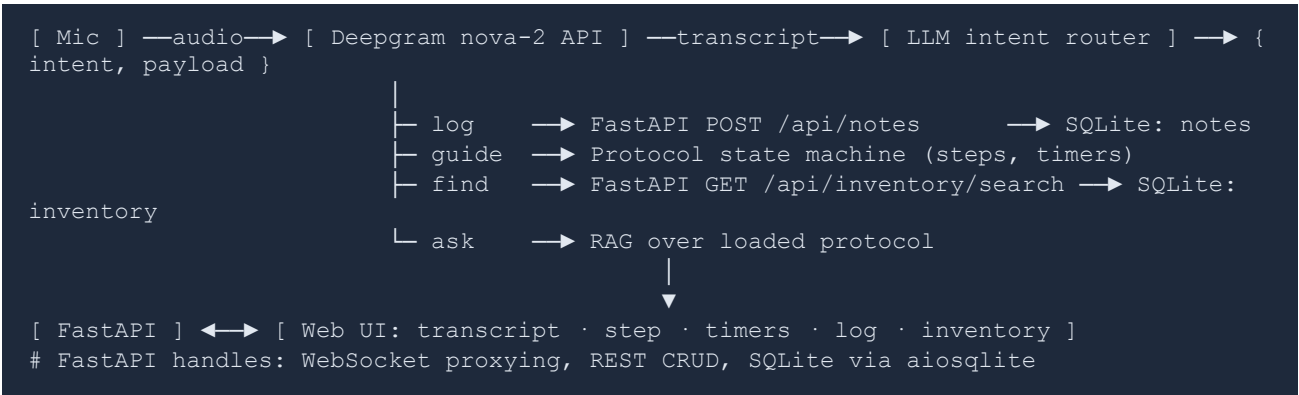
Work is sequenced into four tiers. Each tier gates the next: must-have ships first; nice-to-have begins only once must-have is stable and demo-ready.

Tier	Features	Scope	Gate
Must-have	Voice capture · Deepgram STT · Log · Guide · Timer	Complete, demoable product — the 30-second pitch works at this tier alone. Timer is now must-have.	Ship first
Nice	Find (inventory query)	Adds the generalized lab tool story: 'ELN knows where your reagents are.'	Must-have stable + demo clean
Stretch	Reproducibility flags · Methods export · OCR import · AI voice assistant	Long-tail researcher value. Only pursue if ahead of schedule.	Nice tier complete
Reserve	Polish · Demo rehearsal	Time explicitly protected for final hardening and run-throughs. Never let feature work consume this slot.	Always last

3. System Architecture

3.1 High-Level Pipeline

The system is a linear pipeline from microphone to database, with a fan-out at the intent layer to four handlers and a final convergence at the FastAPI / UI layer.



3.2 Component Descriptions

3.2.1 Voice Capture

Browser MediaStream API captures audio from the default microphone. Audio is streamed over a WebSocket to the Deepgram endpoint via the FastAPI proxy; no local recording file is produced. Interim transcripts are displayed immediately for feedback; final transcripts trigger intent routing.

Property	Value
Protocol	WebSocket (wss://)
Audio encoding	Linear PCM 16-bit, 16 kHz mono
Interim results	Yes — display only, not routed to LLM
Final trigger	is_final: true from Deepgram

3.2.2 Deepgram Transcription

Deepgram's nova-2 model is used for its low-latency streaming transcription. Smart formatting is enabled to handle numeric quantities and units of measure.

 **Scientific vocabulary:**
Common lab terms (microliters, proteinase K, lysis buffer, PCR, etc.) must be added to Deepgram's custom vocabulary list. See §3.3.6 for the full keyword table.

3.2.3 LLM Intent Router

A single structured LLM call converts the transcript into a typed intent and payload. One call keeps latency low and avoids multi-step chained prompts.

Intent	Trigger phrase pattern	Payload fields
log	'Log:', 'Note:', past-tense action	{ text, timestamp, sample_id?, step_ref? }
guide	'Load protocol', 'Next step', 'Start timer'	{ action: 'load' 'next' 'prev' 'timer' 'repeat', protocol_name?, duration_s? }
find	'Where is', 'Where's', 'Location of'	{ reagent_name, fuzzy: true }
ask	'What's next', 'How much', open question	{ question, context: [step_ids] }

3.2.4 Protocol State Machine (Guide Handler)

A protocol is a YAML or JSON file listing ordered steps. When loaded, the state machine tracks the current step index and any active timers. The state machine is a singleton per active session — only one protocol loaded at a time in v1.

3.2.5 SQLite Database

SQLite is used for both persistent stores. A single .db file contains four tables (notes, inventory, protocols, protocol_steps). All access is mediated through FastAPI using aiosqlite. WAL mode is enabled at startup.

3.2.6 FastAPI Backend

FastAPI serves as the thin orchestration layer between handlers, SQLite, and the web UI. It exposes WebSocket endpoints for real-time push and REST endpoints for data loading and queries.

Endpoint	Method / Type	Purpose
<code>/ws/audio</code>	WebSocket	Receive audio from browser, proxy to Deepgram
<code>/ws/events</code>	WebSocket	Push state updates (step, timer, log, inventory) to UI
<code>/api/protocols</code>	GET	List available protocols
<code>/api/protocols/{id}</code>	GET	Load protocol steps into state machine
<code>/api/notes</code>	GET / POST	Fetch paginated log entries / insert new log entry

Endpoint	Method / Type	Purpose
<code>/api/inventory/search</code>	GET	FTS5 fuzzy-match reagent name, return location + qty
<code>/api/inventory</code>	POST	Add or update inventory item
<code>/api/db/status</code>	GET	Health check: row counts, file size, last write

3.2.7 Web UI

A single-page app. All real-time data arrives via the `/ws/events` WebSocket. The UI has five primary panels, all updating without page refresh.

Panel	Content & Behaviour
Live transcript	Scrolling text showing interim (gray) and final (black) transcript. Auto-scrolls to bottom.
Current step	Large text displaying active protocol step. Previous step dimmed above; next step dimmed below.
Timers	One card per active timer. Shows label, countdown (MM:SS), progress ring. Rings amber at 60 s, red at 0.
Log feed	Reverse-chronological list of log entries. Each entry shows timestamp, text, and linked step if present.
Inventory	Read-only view of last find result. Location and approximate quantity displayed prominently.

3.3 Deepgram Speech-to-Text — Integration Protocols NEW

This section defines the required protocols, configuration parameters, and error-handling procedures for the Deepgram nova-2 WebSocket integration.

3.3.1 Connection Protocol

Parameter	Value	Notes
Endpoint	<code>wss://api.deepgram.com/v1/listen</code>	Deepgram Streaming STT
Authentication	<code>Authorization: Token <DEEPGRAM_API_KEY></code>	Stored in server-side env var only. Never exposed to browser.
Model	<code>nova-2</code>	Best balance of accuracy on scientific vocabulary and latency

Parameter	Value	Notes
Encoding	linear16	Linear PCM 16-bit
Sample rate	16000	16 kHz mono
Chunk interval	≤ 100 ms	Browser sends audio chunks every ≤100 ms via /ws/audio

3.3.2 Query Parameters (WebSocket URL)

```
wss://api.deepgram.com/v1/listen
?model=nova-2
&language=en-US
&encoding=linear16
&sample_rate=16000
&channels=1
&interim_results=true
&smart_format=true
&punctuate=true
&utterance_end_ms=1000
&vad_events=true
&keywords=microliters:2&keywords=proteinase:2&keywords=lysis:2
&keywords=PCR:2&keywords=centrifuge:2&keywords=pipette:2
&keywords=aliquot:2&keywords=buffer:1&keywords=incubate:1
```

3.3.3 Message Protocol — Incoming (Deepgram → Server)

Field	Type	Action
type: "Results"	JSON object	Core transcript message. Inspect is_final flag.
is_final: false	boolean	Display interim text in transcript panel (gray); do NOT route to LLM.
is_final: true	boolean	Route to LLM intent router within 200 ms. Display as final (black text).
type: "UtteranceEnd"	JSON object	Signals end of utterance. Flush any pending interim transcript to router if not already routed.
type: "Error"	JSON object	See §3.3.5 Error Handling.

3.3.4 Message Protocol — Outgoing (Server → Deepgram)

Message	Format	When
Audio chunk	Binary (raw PCM bytes)	Every ≤ 100 ms while microphone is active

Message	Format	When
Keep-alive	<pre>{ "type": "KeepAlive" }</pre>	Every 8 seconds when no audio is being sent
Close stream	<pre>{ "type": "CloseStream" }</pre>	When user stops recording session

3.3.5 Error Handling & Reconnection

- WebSocket disconnect (code 1006): exponential backoff reconnect — 500 ms, 1 s, 2 s, 4 s — up to 4 retries before surfacing error to UI.
- Deepgram error response (type: 'Error'): log error type and description; surface human-readable message in transcript panel.
- Auth failure (HTTP 401 on upgrade): fail fast; surface 'Transcription unavailable — check API key'. Do not retry.
- Silence timeout (> 60 s no audio): send KeepAlive or close and re-open WebSocket on next utterance.

3.3.6 Custom Vocabulary (Keywords)

Domain-specific lab terms must be registered as Deepgram keywords at connection time to reduce word-error rate on scientific vocabulary.

Keyword	Boost	Rationale
microliters / μ L	2	Volume units frequently misheard
proteinase K	2	Enzyme name, no common phonetic match
lysis buffer	2	Common reagent, unusual word pair
PCR	2	Acronym, requires explicit boost
centrifuge	2	Uncommon in general STT models
pipette	2	Often mangled to 'pipet' or 'pipe it'
aliquot	2	Technical term, rare phoneme sequence
incubate / incubation	1	Moderately common, slight boost
buffer	1	Common word, minor disambiguation boost



Keyword list maintenance:
Update keywords when new protocols

Keyword	Boost	Rationale
	are loaded that introduce novel reagent or equipment names. Consider making keywords dynamically derived from the active protocol's step text at load time.	

3.4 SQLite via FastAPI — Data Layer UPDATED

All database access is mediated through FastAPI's async request lifecycle. No component other than FastAPI may write to the SQLite file directly at runtime.

3.4.1 Database Initialization Protocol

On FastAPI startup (lifespan context manager), the application shall:

1. Resolve the SQLite file path from the ELN_DB_PATH environment variable (default: ./eln.db).
2. Open a connection usingaiosqlite (async SQLite driver).
3. Execute CREATE TABLE IF NOT EXISTS for all required tables (notes, inventory, protocol_steps, protocols).
4. Enable WAL mode: PRAGMA journal_mode=WAL; for improved concurrent read performance.
5. Enable foreign keys: PRAGMA foreign_keys=ON;
6. Log DB path and row counts to server stdout.

3.4.2 Full SQLite Schema

```
-- Run once at startup via CREATE TABLE IF NOT EXISTS

CREATE TABLE IF NOT EXISTS notes (
  id          INTEGER PRIMARY KEY AUTOINCREMENT,
  text        TEXT      NOT NULL,
  timestamp   DATETIME DEFAULT CURRENT_TIMESTAMP,
  sample_id   TEXT,
  step_ref    INTEGER REFERENCES protocol_steps(id)
```



```

);

CREATE TABLE IF NOT EXISTS inventory (
    id            INTEGER PRIMARY KEY AUTOINCREMENT,
    name          TEXT      NOT NULL,
    location      TEXT,      -- e.g. 'Freezer 2, shelf B'
    quantity_approx TEXT,    -- e.g. '~4 doses'
    notes         TEXT
);

CREATE TABLE IF NOT EXISTS protocol_steps (
    id            INTEGER PRIMARY KEY,
    protocol_id   TEXT      NOT NULL,
    step_number   INTEGER NOT NULL,
    text          TEXT      NOT NULL,
    duration_s    INTEGER,   -- NULL = manual advance
    parameters    TEXT       -- JSON blob of step parameters
);

CREATE TABLE IF NOT EXISTS protocols (
    id            TEXT      PRIMARY KEY, -- e.g. 'dna_extraction'
    name          TEXT      NOT NULL,
    version       TEXT,
    loaded_at     DATETIME DEFAULT CURRENT_TIMESTAMP
);

-- Full-text search virtual table for inventory fuzzy matching
CREATE VIRTUAL TABLE IF NOT EXISTS inventory_fts
    USING fts5(name, location, content='inventory', content_rowid='id');

```

3.4.3 FastAPI Dependency Injection

```

# db.py - Shared async database dependency
import aiosqlite, os

DB_PATH = os.getenv('ELN_DB_PATH', './eln.db')

async def get_db() -> aiosqlite.Connection:
    async with aiosqlite.connect(DB_PATH) as db:
        db.row_factory = aiosqlite.Row
        await db.execute('PRAGMA journal_mode=WAL')
        await db.execute('PRAGMA foreign_keys=ON')
        yield db

# Usage in route handlers:
# @app.post('/api/notes')
# async def create_note(note: NoteIn, db=Depends(get_db)):
#     await db.execute('INSERT INTO notes(text, sample_id, step_ref) VALUES (?, ?, ?)',
#                     (note.text, note.sample_id, note.step_ref))
#     await db.commit()

```

3.4.4 Data Integrity Protocol

- All INSERT and UPDATE operations must use parameterized queries — no string interpolation (SQL injection prevention).
- All writes must call `await db.commit()` immediately after execution. No buffered or deferred commits.
- The server shall return HTTP 500 with a structured error body if any SQLite write fails, and shall log the exception with full traceback.
- The notes table must never be truncated or deleted via the API in v1.

4. Required System Protocols

System-level behavioral specifications. These are distinct from lab experiment protocols (YAML files).

4.1 Audio Streaming Protocol

Parameter	Required Value
Transport	WebSocket (<code>wss://</code>) — server to Deepgram; <code>ws://</code> or <code>wss://</code> browser to server
Audio format	Linear PCM, 16-bit, 16 kHz, mono (1 channel)
Chunk size	≤ 100 ms of audio per send (1,600 samples per chunk at 16 kHz)
Browser API	MediaStream API + ScriptProcessorNode or AudioWorkletNode
Silence handling	Continue sending audio (Deepgram VAD handles silence detection)
Stop command	Send <code>CloseStream</code> JSON message; do not abruptly close WebSocket

4.2 Transcript Routing Protocol

- **Interim results** (`is_final: false`): Broadcast to UI via `/ws/events` for live display only. Must NOT be sent to LLM.
- **Final results** (`is_final: true`): Must be dispatched to the LLM intent router within 200 ms.
- **Empty transcripts**: Discard silently. Do not route empty strings to LLM.
- **Duplicate prevention**: Use Deepgram's `speech_final` flag in conjunction with `is_final` to avoid routing the same utterance twice.

4.3 Intent Routing Protocol

- Each final transcript must produce exactly one LLM call with the structured output schema defined in §3.2.3.
- LLM calls must use JSON mode (structured output) to guarantee parseable responses.
- If the LLM returns a malformed response, default to intent: 'ask' with the original transcript as the question.
- LLM API key stored server-side in environment variable only (ANTHROPIC_API_KEY or OPENAI_API_KEY). Never exposed to browser.
- Maximum LLM response wait: 1,500 ms. If exceeded, surface a timeout response to the UI.

4.4 State Machine Transition Protocol

- The protocol state machine must be a singleton per active session — only one protocol loaded at a time in v1.
- Step transitions must be broadcast to all connected UI clients via /ws/events within 50 ms of the handler completing.
- Timer state (countdown value) must be broadcast every 1 second $\pm$ 50 ms.
- Timer expiry must trigger both an audible alert (Web Audio API tone) and a visual indicator (red ring) simultaneously.
- Loading a new protocol clears all active timers from the previous protocol.

4.5 WebSocket Events Protocol (/ws/events)

All server-to-client pushes follow a typed event envelope:

```
{
  "event": "transcript" | "step_change" | "timer_tick" | "timer_expire"
           | "log_entry" | "inventory_result" | "error",
  "payload": { ... },
  "timestamp": "2026-06-20T13:30:00Z"
}
```

Event type	Payload fields	Trigger
transcript	text, is_final	Every Deepgram result message
step_change	prev_step, current_step, next_step	Guide handler advances/retreats cursor
timer_tick	timer_id, label, remaining_s	Every 1 second while timer active
timer_expire	timer_id, label	When remaining_s reaches 0
log_entry	id, text, timestamp, sample_id, step_ref	After successful SQLite INSERT into notes

Event type	Payload fields	Trigger
<code>inventory_result</code>	name, location, quantity_approx	After inventory fuzzy search
<code>error</code>	code, message, source	Any handler or DB error

4.6 Security Protocol

- All API keys (Deepgram, LLM provider) must be stored in server-side environment variables. Never hardcoded or returned via any API endpoint.
- The `/api/db/status` endpoint must not expose the full filesystem path in production builds.
- WebSocket connections from the browser must be validated to originate from localhost in development.
- No user authentication is required in v1 (single-user, local use only).

4.7 Offline Operation Protocol

- After initial protocol and inventory load, the system must remain operational for all non-transcription functions (guide, log, timer) if the network is lost.
- Deepgram and LLM calls require an active network connection. The system shall surface 'Transcription unavailable — network required' when offline.
- SQLite reads and writes must continue to function fully offline.

5. Frontend Requirements



New in v0.3:

Frontend page structure, navigation shortcuts, and protocol upload flow added based on product review.

5.1 Page Structure

The frontend is a single-page app with three primary views navigated by shortcuts:

Page / View	Description
Main (Dashboard)	Default landing page. Shows live transcript, current step, timers, log feed, inventory panel. Protocol assistant shortcut visible when a protocol is active.

Page / View	Description
Protocol Library	Lists all uploaded protocols with a shortcut card for each. Shows 'No protocols uploaded' with an upload prompt if empty. Each card links to the Protocol Detail page.
Protocol Detail	Shows full protocol with all steps, reagent summary, and a Run button. Tapping Run launches the protocol assistant on the Main page.
Protocol Upload	Upload flow for new protocols. Accepts .docx, .yaml, .pdf. Presents a parsed summary page before confirming upload.
Lab Inventory	Full inventory view. Add, edit, and search reagents. Displays location, quantity, and notes. Synced with voice find intent.

5.2 Navigation Shortcuts

Persistent shortcuts are accessible from all pages (header or sidebar). When a protocol is active, the assistant shortcut is always visible.

- **Assistant shortcut:** Opens automatically when a protocol is started via voice. Also accessible manually. Brings the protocol assistant overlay to focus.
- **Protocol library shortcut:** Direct link to the protocol list. If no protocols are loaded, shows prompt to upload.
- **Lab inventory shortcut:** Direct link to the inventory management page.
- **Run protocol shortcut:** On the Protocol Detail page only. Opens the protocol assistant on the main page and starts voice guidance.

5.3 Protocol Upload Flow NEW

The upload flow is designed to let researchers correct parsing errors before a protocol is committed to the database.

Step	Screen	Detail
1	Upload page	User selects or drags a .docx, .yaml, or .pdf protocol file.
2	Parsing	FastAPI + LLM extract steps, parameters, reagent names, and volumes from the uploaded file.
3	Protocol summary page	Shows parsed steps in an editable list. Includes a running reagent total (quantity × number of samples). Reagents can be grouped to handle synonyms (e.g. 'EDTA' and 'Ethylenediaminetetraacetic acid' mapped to the same item). User can edit any step or reagent value before confirming.

Step	Screen	Detail
4	Confirm	User clicks 'Protocol uploaded' / confirm. Protocol is written to SQLite and appears as a shortcut on the Protocol Library page.
5	Protocol Library	New protocol card is visible. Shortcut links to Protocol Detail page with a Run button.



Reagent synonym grouping:

The summary page must surface cases where the LLM extracted the same reagent under two different names (e.g. full IUPAC name vs abbreviation). The researcher approves the grouping before upload. This directly improves voice find accuracy.

5.4 Protocol Guidance View (Assistant)

The main protocol assistant is the core user experience. It opens automatically when a protocol is loaded and is the primary surface for voice interaction.

- **Step display:** Current step large and prominent. Previous step dimmed above, next step dimmed below. Step number shown.
- **Voice guidance:** Primarily used for asking questions ('How many mL is needed for 3 samples?') and getting the next step. LLM answers using the loaded protocol as context (RAG).
- **Timer integration:** Active timers shown inline within the assistant view. Audible alert on expiry.
- **Edit mode:** Protocol editing does not require voice recognition — editing steps and values is done via the Protocol Detail page before running.

6. Functional Requirements

Requirements are prefixed FR and labeled by tier.

6.1 Voice Capture and Transcription (must-have)

ID	Tier	Requirement
FR-01	Must	The system shall request microphone permission on first use and remember the grant for the session.
FR-02	Must	Audio shall be streamed to Deepgram via WebSocket at ≤ 100 ms chunk intervals.

ID	Tier	Requirement
FR-03	Must	Interim transcripts shall appear in the transcript panel within 300 ms of the first word being spoken.
FR-04	Must	Final transcripts shall be routed to the intent router within 200 ms of Deepgram emitting is_final: true.
FR-05	Must	The system shall remain in active-listening mode between utterances without manual reactivation.
FR-05 a	Must	The FastAPI server shall proxy all audio to Deepgram. The browser shall never communicate directly with Deepgram.
FR-05 b	Must	The Deepgram connection shall send a KeepAlive message every 8 seconds when no audio is being transmitted.
FR-05 c	Must	On Deepgram WebSocket disconnect, the server shall attempt reconnection with exponential backoff (up to 4 retries) before surfacing an error.

6.2 LLM Intent Routing (must-have)

ID	Tier	Requirement
FR-06	Must	The intent router shall classify each utterance into exactly one of: log, guide, find, ask.
FR-07	Must	The router shall return a structured JSON payload conforming to the per-intent schema defined in §3.2.3.
FR-08	Must	End-to-end latency from final transcript to spoken system response shall be ≤2 s on a standard laptop with broadband.
FR-09	Must	Unrecognized or ambiguous utterances shall be classified as ask with an appropriate clarification response.

6.3 Log (must-have)

ID	Tier	Requirement
FR-10	Must	Each log utterance shall create a row in the notes table within 500 ms of intent classification, via POST /api/notes.
FR-11	Must	The log feed panel shall update in real time without page refresh (via /ws/events log_entry).
FR-12	Must	Each log entry shall record: free text, UTC timestamp, current step ID (if a protocol is active), and sample ID if parseable from the utterance.

ID	Tier	Requirement
FR-13	Must	Log entries shall be persisted to SQLite and survive a browser refresh or server restart.

6.4 Guide / Protocol State Machine (must-have)

ID	Tier	Requirement
FR-14	Must	The system shall accept 'Load [protocol name]' and initialize the state machine at step 1.
FR-15	Must	'Next step', 'Next', or equivalent shall advance the cursor and read the new step aloud via TTS.
FR-16	Must	'Start a [N]-minute timer' / 'Start a [N]-second timer' shall create a named countdown timer visible in the timers panel.
FR-17	Must	Timer expiry shall produce an audible alert and a visual indicator even if the user is not looking at the screen.
FR-18	Must	Protocol steps with a non-null duration_s shall auto-start a timer when the step is activated.
FR-19	Must	The current step panel shall always show the active step prominently, with the previous and next steps dimmed for context.

6.5 Frontend Pages & Upload (must-have) NEW

ID	Tier	Requirement
FR-24	Must	The frontend shall have a Protocol Library page listing all uploaded protocols with shortcut cards.
FR-25	Must	Each protocol card shall link to a Protocol Detail page with a Run button that launches the voice assistant.
FR-26	Must	The protocol upload page shall accept .docx, .yaml, and .pdf files and present a parsed summary for review before saving.
FR-27	Must	The protocol summary page shall display a running reagent total and allow step values to be edited before confirming upload.
FR-28	Must	The summary page shall surface reagent synonym conflicts (e.g. EDTA vs Ethylenediaminetetraacetic acid) and allow the researcher to group them before upload.

6.6 Find / Inventory (nice-to-have)

ID	Tier	Requirement
FR-20	Nice	'Where is [reagent]?' shall fuzzy-match the reagent name against the inventory table and return location and approximate quantity.
FR-21	Nice	The inventory panel shall update to show the most recent find result with reagent name, location, and quantity.
FR-22	Nice	Inventory data shall be pre-loadable from a CSV file on server startup via POST /api/inventory.
FR-23	Nice	If no match is found, the system shall respond 'I don't have a record for [reagent]' rather than silently failing.

6.7 Stretch Features

- **Reproducibility flags:** Automatically flag log entries that deviate from protocol step parameters.
- **Methods export:** Export a formatted lab methods section from the completed log and protocol (DOCX or PDF).
- **OCR import:** Photograph a printed protocol and import via image upload + LLM vision extraction.
- **AI voice assistant:** A more capable conversational assistant mode that can answer complex lab questions, suggest optimizations, and flag potential issues with the protocol — beyond the current intent-routing model. NEW in v0.3.



None of the stretch features are required for the demo. Begin only if must-have and nice tiers are complete and stable with time remaining.

7. Non-Functional Requirements

NFR	Category	Requirement
Latency	Performance	End-to-end voice-to-response ≤ 2 s. Transcription interim ≤ 300 ms. Timer tick ≤ 50 ms drift.
Reliability	Availability	System shall operate offline after initial protocol and inventory load (Deepgram and LLM require network).
Browser support	Compatibility	Chrome 120+ and Edge 120+. Firefox excluded (MediaStream reliability). Safari excluded (WebSocket/Audio API limitations).
Data persistence	Durability	All log entries persisted to SQLite on write via FastAPI. No data loss on server crash after commit.

NFR	Category	Requirement
Security	Auth	API keys (Deepgram, LLM) stored server-side in environment variables only. Never exposed to browser.
Accessibility	A11y	Timer alerts must be audible. Key state (current step, active timer) must be readable without vision (ARIA live regions).

8. Demo Script — 30-Second Pitch

The demo is voice-only. Hands never touch the machine. Each beat is listed with the expected system response and the intent / handler that produces it.



Rehearsal time is explicitly reserved in the build tier schedule. Run the full demo at least 5 times before the actual pitch to catch edge cases in transcription and LLM routing.

#	Spoken utterance	Expected response	Intent	Handler
1	"Load DNA extraction protocol."	"Step 1: add 200 µL lysis buffer." — step panel updates, TTS reads step.	guide	State machine: load
2	"Log: added 200 microliters lysis buffer to sample A."	Structured entry appears in log feed with timestamp and step reference.	log	POST /api/notes → SQLite
3	"Start a 10-minute incubation timer."	Timer card appears in timer panel. Countdown begins at 10:00.	guide	State machine: timer
4	"Where's the proteinase K?"	"Freezer 2, shelf B, approximately 4 doses left." Inventory panel updates.	find	GET /api/inventory/search → FTS5
5	"What's next?"	Next step read aloud via TTS. Step panel advances.	ask	RAG over protocol

9. Data Models

9.1 Protocol File Format

```
# dna_extraction.yaml
protocol:
  id: dna_extraction
```

```

name: DNA Extraction
version: 1.0
steps:
  - id: 1
    text: Add 200 µL lysis buffer to sample tube.
    duration_s: null
    parameters:
      volume_ul: 200
      reagent: lysis buffer
  - id: 2
    text: Incubate at 65°C for 10 minutes.
    duration_s: 600
    parameters:
      temp_c: 65
      duration_s: 600

```

9.2 Inventory CSV Format

```

name,location,quantity_approx,notes
Proteinase K,Freezer 2 shelf B,~4 doses,10 mg/mL aliquots
Lysis buffer,Fridge 1 shelf A,~200 mL,
Ethanol 70%,Cabinet 3,2 bottles,
EDTA,Fridge 1 shelf B,~50 mL,see also: Ethylenediaminetetraacetic acid

```

9.3 LLM Intent Schema (JSON)

```

{
  "intent": "log" | "guide" | "find" | "ask",
  "payload": {
    // log
    "text": string,
    "sample_id": string | null,
    "step_ref": integer | null,
    // guide
    "action": "load" | "next" | "prev" | "timer" | "repeat",
    "protocol_name": string | null,
    "duration_s": integer | null,
    "timer_label": string | null,
    // find
    "reagent_name": string,
    // ask
    "question": string,
    "context_step_ids": integer[]
  }
}

```

10. Open Questions & Decisions Needed

#	Question	Options	Owner
1	Which LLM for intent routing? Needs structured output, low latency, low cost per call.	Claude claude-haiku-4-5 · GPT-4o-mini · Gemini Flash	Engineering lead
2	TTS provider for spoken step responses? Must work in-browser with minimal latency.	Deepgram Aura · ElevenLabs · Web Speech API (free)	Engineering lead
3	Protocol file storage: server filesystem vs. user-uploadable at runtime?	Filesystem (simple) · Upload endpoint (flexible)	Product owner
4	RAG retrieval for 'ask' intent: embed steps at load time or do keyword search?	Embedding + vector search · SQLite FTS5 keyword search	Engineering lead
5	Demo environment: local laptop only, or should it work on conference room projector wifi?	Local only (simplest) · Cloud hosted (riskier, failover needed)	Product owner
6	Deepgram keyword list: static at startup or dynamically derived from active protocol?	Static env config · Dynamic from protocol YAML on load	Engineering lead
7	SQLite file backup strategy for demos: copy-on-start snapshot or none?	No backup (v1) · Auto-copy to eln_backup_{date}.db on startup	Engineering lead
8	Protocol upload parsing: use LLM vision (for PDFs/scans) or structured YAML only for v1?	YAML only (simpler) · LLM extraction (more flexible, needed for .docx)	Engineering lead
9	Reagent synonym grouping: automatic (LLM suggests) or fully manual on the summary page?	Manual only (safer) · LLM-suggested with human approval	Product owner

11. Out of Scope (v1)

- Multi-user collaboration — no shared sessions or concurrent users in v1
- Authentication and authorization — single-user, local use only
- Mobile / tablet support — desktop Chrome/Edge only
- Cloud sync or backup of the SQLite database
- Integration with LIMS (Laboratory Information Management System)
- Automated protocol compliance reporting or audit trails
- Support for protocols longer than 50 steps (no artificial limit, but untested)

12. Appendix — Technology Stack

Layer	Technology	Notes
Audio capture	Browser MediaStream API	Built-in, no library needed
Transcription	Deepgram nova-2 (WebSocket)	~\$0.0059/min. Custom vocabulary for lab terms. Proxied via FastAPI.
Intent routing	Anthropic / OpenAI (structured output)	Single call per utterance. JSON mode required.
Backend	FastAPI (Python)	Async WebSocket support built-in. Handles Deepgram proxy, SQLite, event broadcast.
Database	SQLite via aiosqlite + FastAPI	File-based, no server process. WAL mode enabled. All access mediated through FastAPI.
Frontend	Vanilla JS + HTML (or React)	Vanilla JS minimizes build tooling complexity for hackathon.
TTS	TBD (see open question #2)	Web Speech API is zero-cost but voice quality varies.
Protocol parsing	LLM vision + structured extraction	Required for .docx and .pdf upload. YAML files parsed directly.



Key architectural decisions:

(1) Deepgram is proxied exclusively through FastAPI — API key never touches the browser. (2) SQLite is the single source of truth for all persistent data — all reads and writes go through FastAPI endpoints using aiosqlite. (3) WebSocket event bus (/ws/events) decouples all UI updates from HTTP request/response cycles. (4) Protocol upload uses LLM extraction with human review before any data is committed.

— End of document —